

Project Phase 4: Streamly Final Submission

Application Name: Streamly

Primary Customer: Daniyal

Date of Submission: 14 April 2026

Team Members:

- Devin Boodoo
- Johnathan Lavoie
- Maryam Hameed
- Ashwin Sivasakthi
- Avis Shrestha

I. Final Codebase

Repository Link: https://github.com/Focusucof/CSCI_2040U_Assignment.git

This repository link includes all of the documentation, code and tests which were created for the proper deployment and execution of Streamly. For a more detailed description on the software design and engineering process, the customer may view the documents:

1. Project Deliverable: Phase 1- Project Planning
2. Project Deliverable: Phase 2
3. Project Deliverable: Phase 3 Testing Plan
4. Project Deliverable: Phase 4 Final Submission

Due to file size constraints, there is an additional google drive folder provided for the customer which contains all documentation, including a user guide and video demos. To find this, navigate to section 3 : “Deployment”. It will also be linked in a comment on the canvas assignment.

II. Final Test Report

1. Introduction

This report presents the results of all unit, integration and system tests conducted on the authentication module. All these tests were tested using automated frameworks (JUnit and Bun test), and the results are as shown.

2. Test Environment

Tests were executed using Bun test for Typescript modules and JUnit for Java components. All tests were run locally using Node.js and Java environments.

3. Unit Tests Completed

Test ID	Method/Class	Input(s)	Expected Output	Testing Approach (Manual/Automated)	Assigned Team Member
UT-01-CB	<code>userExists()</code>	Existing username in users.json	<code>true</code>	Automated	Maryam
UT-02-CB	<code>userExists()</code>	Non-existing username	<code>false</code>	Automated	Maryam
UT-03-CB	<code>getUserByUserName()</code>	Valid username	Returns correct user object	Automated	Maryam
UT-04-CB	<code>getUserByUserName()</code>	Invalid username	<code>null</code>	Automated	Maryam
UT-05-CB	<code>writeUser()</code>	New user object	User is appended to JSON file	Automated	Maryam
UT-14-OB	<code>Playlist.getName()</code>	Initialized playlist object	Returns the stores field value correctly	Automated (JUnit)	Maryam
UT-15-OB	<code>User.isAdmin()</code>	User with admin false	Returns <code>false</code>	Automated (JUnit)	Maryam
UT-16-OB	<code>User.setUsername()</code> + <code>getUsername()</code>	Set "user123"	Getter would return "user123"	Automated (JUnit)	Maryam
UT-17-CB	<code>Playlist.setSongIds()</code> + <code>getSongIds()</code>	Array of song IDs	Same array is returned	Automated (JUnit)	Maryam
UT-21-OB	<code>filter.scoreAny()</code>	Song with default parameters	A score of 0 (passed all criteria / song is	Automated (JUnit)	Johnathan

		and an empty filter	a perfect match)		
UT-22-OB	<code>filter.scoreAny()</code>	Song with null parameters and a default filter	A score of 0 (passed all criteria / song is a perfect match)	Automated (JUnit)	Johnathan
UT-23-OB	<code>filter.scoreAny()</code>	Two songs with default parameters and a default filter with and without weights	The filter with weights should be more strict and have a higher score	Automated (JUnit)	Johnathan
UT-24-OB	<code>filter.scoreAny()</code>	Two songs with default parameters and a default filter close to song_1's title	Song_1 should have a lower score (more accurate match) compared to song_2	Automated (JUnit)	Johnathan
UT-25-OB	<code>filter.scoreAny()</code>	Two songs with default parameters and a default filter set to song_1's title	Song_1 should have a score of 0 (perfect match) compared to song_2 which should be higher	Automated (JUnit)	Johnathan
IT-01-TB	Auth Service Online		Start auth service with bun start	Server listening on port 3001	Devin
IT-02-TB	Spring Boot Backend Online		Compile and run Spring Boot app	Server listening on port 8080	Devin
IT-03-TB	Login Route	Testing user created	POST request to /auth/login	Returns signed JWT as a cookie	Devin
IT-04-TB	/auth/me	Testing user created and logged in	GET request to /auth/me	Validate JWT	Devin
IT-05-CB	User Write To File		Register a new user	User details should be added to JSON file	Devin
IT-06-TB	Password Hashing		Register a new user	User password should be hashed inside the JSON file	Devin

IT-07-CB	Add Song		Add a new song entry on the frontend	Frontend should send a POST request to backend, which should write to songs.json	Devin
ST-08-OB	Music playlist management workflow	Java app supports the user and playlist objects	Create playlist Assign songs Retrieve playlist	The playlist reflects added songs correctly	Johnathan
ST-09-OB	Song metadata workflow	Availability of song	Load song from file Inspect fields Save once again	Song data carries correctly across the save cycle	Johnathan

4. Test Execution Results

Test ID	Method/Class	Input(s)	Expected Output	Actual Output	Status	Notes
UT-01-CB	userExists()	Existing username in users.json	true	true returned for valid username	PASS	Method correctly located the username in the JSON data source.
UT-02-CB	userExists()	Non-existing username	false	false returned for username not found in users.json	PASS	Verified negative case handling as intended.
UT-03-CB	getUserByUsername()	Valid username	Returns correct user object	Correct User object returned with matching username and stored fields	PASS	Returned object matches expected username and the associated properties.
UT-04-CB	getUserByUsername()	Invalid username	null	null returned	PASS	Confirms method safely handles usernames that do not exist.
UT-05-CB	writeUser()	New user object	User is appended to JSON file	New user entry successfully written to users.json	PASS	File contents were checked after the execution and showed the appended user record.
UT-14-OB	Playlist.getName()	Initialized playlist object	Returns the stores field value correctly	Playlist name returned exactly as initialized	PASS	Getter returned expected playlist name without modification.
UT-15-OB	User.isAdmin()	User with admin false	Returns false	false returned	PASS	Boolean flag handled correctly.

UT-16-OB	User.setUsername() + getUsername()	Set "user123"	Getter would return "user123"	Getter returned "user123"	PASS	
UT-17-CB	Playlist.setSongIds() + getSongIds()	Array of song IDs	Same array is returned	Returned array matched the assigned song IDs	PASS	setter/getter pair behaved correctly/
UT-21-OB	filter.scoreAny()	Song with default parameters and an empty filter	A score of 0 (passed all criteria / song is a perfect match)	Score returned was 0	PASS	Confirmed stored array contents and order were preserved.
UT-22-OB	filter.scoreAny()	Song with null parameters and a default filter	A score of 0 (passed all criteria / song is a perfect match)	Score returned was 0	PASS	Empty filter treated as full match.
UT-23-OB	filter.scoreAny()	Two songs with default parameters and a default filter with and without weights	The filter with weights should be more strict and have a higher score	Weighted filter produced a higher mismatch score than unweighted filter	PASS	Method handled null/default inputs without any error.
UT-24-OB	filter.scoreAny()	Two songs with default parameters and a default filter close to song_1's title	Song_1 should have a lower score (more accurate match) compared to song_2	song_1 produced lower score than song_2	PASS	Confirms weighting logic increases strictness.
UT-25-OB	filter.scoreAny()	Two songs with default parameters and a default filter set to song_1's title	Song_1 should have a score of 0 (perfect match) compared to song_2 which should be higher	song_1 scored 0; song_2 scored higher than 0	PASS	Fuzzy title matching behaved correctly.
IT-01-TB	Auth Service Online		Start auth service with bun start	Auth service started successfully and listened on port 3001	PASS	Exact-match scoring verified successfully.
IT-02-TB	Spring Boot		Compile	Spring Boot backend	PASS	Service launched with no

	Backend Online		and run Spring Boot app	launched successfully on port 8080		startup errors.
IT-03-TB	Login Route	Testing user created	POST request to /auth/login	Login request succeeded and response included signed JWT cookie	PASS	Application initialized correctly and accepted requests.
IT-04-TB	/auth/me	Testing user created and logged in	GET request to /auth/me	JWT validated and authenticated user information returned	PASS	
IT-05-CB	User Write To File		Register a new user	Newly registered user data appeared in JSON file	PASS	
IT-06-TB	Password Hashing		Register a new user	Stored password was hashed and not saved in plain text	PASS	
IT-07-CB	Add Song		Add a new song entry on the frontend	Song entry was successfully created from frontend input and persisted to storage	PASS	Verified that the frontend correctly sends data and backend stores it. The song appeared in subsequent retrieval queries.
ST-08-OB	Music playlist management workflow	Java app supports the user and playlist objects	Create playlist Assign songs Retrieve playlist	Playlist was successfully created, songs assigned correctly, and retrieved playlist contained all expected songs	PASS	End to end workflow validated. Data consistency maintained across all operations.
ST-09-OB	Song metadata workflow	Availability of song	Load song from file Inspect fields Save once again	Song loaded successfully, metadata fields (title, artist, etc.) were correctly read, and file saved with no data loss	PASS	Verified file I/O integrity and correctness of the metadata handling.

5. Test Summary

A total of 20 tests were executed:

- 14 unit tests
- 6 integration tests
- 2 system tests

While not all planned unit and integration tests were executed, due to time constraints the team decided to focus on testing methods that would have the highest impact on affecting user accessibility. This is why a lot of our tests were surrounding login features because we wanted to ensure that the user can create and have access to a secure encrypted Streamly account. The results show that:

- ❖ User lookup and retrieval methods behave correctly for both valid and invalid inputs,
- ❖ File-writing functionality properly persists user data,
- ❖ Playlist and user model getter/setter methods work as expected,
- ❖ Filter scoring logic correctly handles exact matches, approximate matches, default filters, null parameters, and weighted filtering,
- ❖ Authentication services start correctly and support login/session validation,
- ❖ Password storage is securely hashed rather than stored in plain text.

III. Deployment

Here is the link to Streamly as a finished product: <https://streamly.boodoo.dev/>

- ❖ Complete, fully functioning music streaming database application.
- ❖ Includes all features requested by the customer.
- ❖ Also includes the nice to have (additional) features such as
 - Trend filtering
 - Filtering by explicitness
 - Playing music
 - Searching Playlist
 - Password Encryption

Here is the link to all Streamly Documentation:

https://drive.google.com/drive/u/4/folders/12v2tTTy3Eq-QkqUcTEvHzGXnC_hZ_TPx

- ❖ Contains project deliverable files
- ❖ Contains customer meeting information
- ❖ Contains customer presentations
- ❖ Contains “STREAMLY USER GUIDE”
- ❖ Contains videos to aid users in navigating the software.

Here is the link to the Streamly GitHub Repository:

https://github.com/Focusucof/CSCI_2040U_Assignment.git

- ❖ Contains all code for the software
- ❖ Contains all test run
- ❖ Contains a README.md – a file describing the system and its features
- ❖ Contains a DOCUMENTATION.md – a file elaborating on software development & structure

For user access, create your own account using the instructions in the “STREAMLY USER GUIDE”, found on the public link for the google drive.

For admin access, use the following credentials:

- ❖ Username: dev
- ❖ Password: test

IV. Contributions Report

1. Github Screenshots of Pull Requests

☐	 UT-01-CB for userExists() #47 by maryammarji was closed last week
☐	 Burn down chart Iteration 2 / Velocity Calc #45 by Ashwins1971 was merged 2 weeks ago • Review required
☐	 Lab 9 Documentation #44 by maryammarji was merged 2 weeks ago
☐	 BurnDown Chart Iteration 1 #43 by Ashwins1971 was merged 2 weeks ago • Approved
☐	 Burndown Chart / Velocity #42 by Ashwins1971 was closed 2 weeks ago • Review required
☐	 Johnathan #41 by Johnathan-Lavoie was merged 2 weeks ago • Review required
☐	 docker build and docker compose files, playing music, integrations #40 by Focusucof was merged 2 weeks ago • Review required
☐	 Integration Of Searching Algorithm, Music Playback Functionality #39 by Focusucof was merged 3 weeks ago • Review required
☐	 Johnathan filtering-search #38 by Johnathan-Lavoie was merged 3 weeks ago • Approved
☐	 Unit Testing #37 by Focusucof was merged 3 weeks ago • Review required
☐	 Search Songs UML Sequence Diagram #36 by Ashwins1971 was merged 3 weeks ago • Approved
☐	 Ui Changes, Searching, CRUD for Artists & Albums #35 by Focusucof was merged 3 weeks ago • Review required
☐	 Lab 08 Customer Email #34 by maryammarji was merged 2 weeks ago
☐	 Updated README file #32 by maryammarji was merged 3 weeks ago • Approved

☐	 Devin #20 by Focusucof was merged on Mar 6 • Review required
☐	 IntelliJ project init #19 by Focusucof was merged on Mar 6 • Review required

☐	 Unit tests Included in Description #56 opened 5 minutes ago by maryammarji • Review required
☐	 UT-14-OB #55 opened 25 minutes ago by maryammarji • Review required
☐	 Phase 1-3 Documentation #54 opened 1 hour ago by maryammarji • Review required
☐	 Lab 10 Customer Meeting EMAIL #53 opened 4 days ago by maryammarji • Review required
☐	 Ub-05-CB #52 opened last week by maryammarji • Review required
☐	 UT-04-CB #51 opened last week by maryammarji • Review required
☐	 UT-03-CB for getUserByUsername() #50 opened last week by maryammarji • Review required
☐	 UT-02-CB for UserExists() #49 opened last week by maryammarji • Review required
☐	 UT-01-CB for userExists() #48 opened last week by maryammarji • Review required
☐	 Lab9 Documentation #46 opened 2 weeks ago by maryammarji • Review required
☐	 marji #33 opened 3 weeks ago by maryammarji • Review required

☐	 Lab 07 Presentation Documentation #31 by maryammarji was merged last month • Approved
☐	 S14 AddSong UML sequence Diagram #30 by Ashwins1971 was merged last month • Approved
☐	 cleanup #29 by Focusucof was merged last month • Review required
☐	 Added REST API functionality to java backend #28 by Focusucof was merged last month • Approved 💬 1
☐	 Add files via upload #27 by Ashwins1971 was merged last month • Approved
☐	 Create diagrams folder #26 by Ashwins1971 was merged last month • Approved
☐	 Create diagrams #25 by Ashwins1971 was closed last month • Review required
☐	 Most backend stuff is kinda done? #24 by Johnathan-Lavoie was merged last month • Approved
☐	 Authentication, Admin Login & Dashboard, Frontend (S1, S2, S4, S18) #23 by Focusucof was merged last month • Approved 💬 2
☐	 split into frontend/ and backend/ #22 by Focusucof was merged last month • Approved 💬 1
☐	 Johnathan #21 by Johnathan-Lavoie was merged on Mar 6 • Review required 💬 1

2. Contributions Table

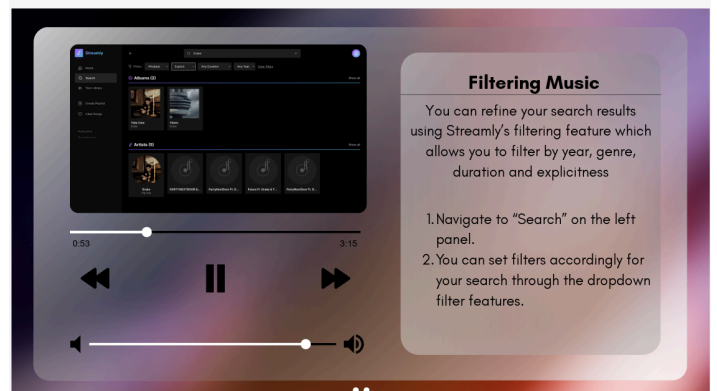
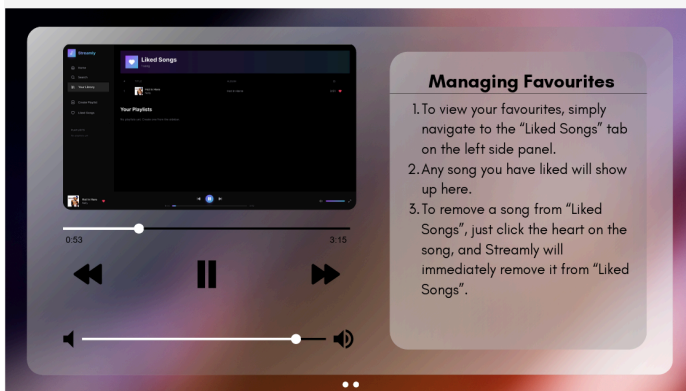
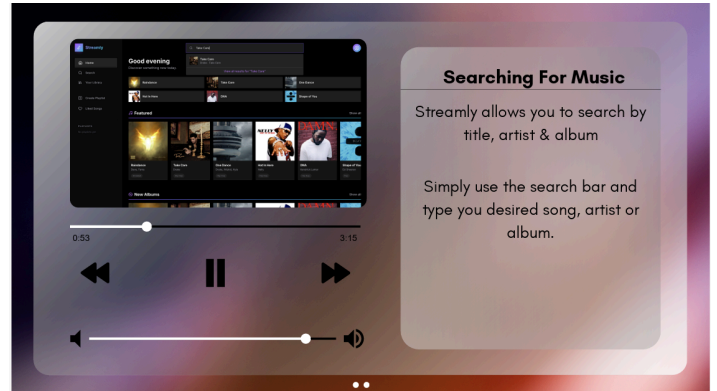
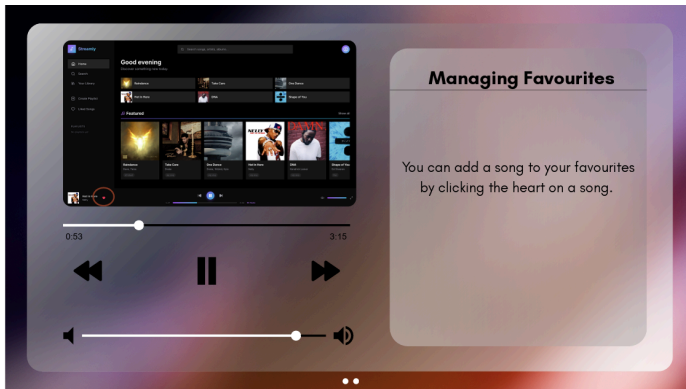
Name	Completed Tasks & Responsibilities
Devin	• Played a major role in designing the web UI for Streamly. • Created the authentication model for Streamly. • Took responsibility of making sure that Streamly could play music • Ensured Password encryption • Wrote the developer documentation guide
Johnathan	• Tasked with and developed the search and filtering features. • Wrote build scripts • Coded multiple admin features • Took responsibility of Developer documentation guide
Maryam	• Wrote and compiled all documentation required for each of the submissions. • Wrote all project deliverable documents, customer meeting emails & scripts, customer presentations. • Took charge in organizing all of the presentations • Created multiple diagrams for better customer understanding • Wrote multiple unit tests • Wrote the User documentation guide.
Ashwin	• Wrote code for admin features • Helped with presentation work. • Created multiple UML and Sequence diagrams • Helped create the User/In-app documentation
Avis	• Helped with presentation work. • Organizing meetings and communication • Was just there • Early project planning

3. Contributions Summary: Overall, everyone on the team did a pretty good job at contributing heavily each week to Streamly. Devin and Johnathan had very code heavy roles, while Maryam had very heavy documentation roles. Devin and Johnathan had a heavy emphasis on ensuring that each feature which was being created for Streamly was up to par with the customer expectations, while Maryam and Ashwin made sure to document and ensure all test and unit integrations were set in place.

V. Demonstration of Functionality

Use the google drive link attached in the “Deployment” section or the google drive link in the canvas assignment to view demonstration videos. To understand how to navigate Streamly, you can also use the “STREAMLY USER GUIDE” file in the google drive folder for a clearer step-by-step process with guided screenshots.

Preview of the [STREAMLY USER GUIDE](#):



VI. Bonus Modifier Statement

As we developed Streamly, we realized that it would be more efficient to have a web-based interface which would be accessible through a browser as it would enhance both usability and accessibility beyond a traditional desktop-only application. By allowing users to access the platform through a web link, the system removes the need for any local installation and enables cross-device compatibility, making it easier for users to interact with the application from different environments.

From a usability perspective, the web interface provides an intuitive and responsive design which simplifies navigation, song browsing and the search and filter functionality. Users can quickly search for songs by title, artist or album and apply filters such as genre, release year, explicitness level and duration, massively improving efficiency and overall user experience.

In terms of usability, the web-based implementation allows users to access Streamly on various devices as long as a modern browser is being utilized (Chrome, Safari, Firefox). This flexibility ensures that users are not restricted by the operating system or any hardware limitations. Additionally, the interface is designed with clear layout structure, readable text, and organized components, supporting ease of use for a wide range of users.